Software Requirements Specification

for

Blog Management System

Version 1.0

Prepared by Md Ismile Hosen Siam

13 January, 2026

Table of Contents

Table of Contents ii

1. Introduction 1

1.1 Purpose 1

1.2 Project Scope 1

1.3 References 2

2. Overall Description 2

2.1 Product Perspective 2

2.2 Product Features 2

2.3 User Classes and Characteristics 3

2.4 Operating Environment 3

2.5 Design and Implementation Constraints 3

2.6 Assumptions and Dependencies 4

3. System Requirements 4

3.1 Functional Requirements 4

3.2 Non-Functional Requirements 11

3.3 Interface Requirements 12

4. System Architecture 13

4.1 Project Structure 13

4.2 Technology Stack 14

Appendix A: Glossary 14

Appendix B: Analysis Models 16

Use Case Diagram 16

Level 0 Data Flow Diagram 16

Level 1 Data Flow Diagram 17

Entity Relationship Diagram 17

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the requirements of the **Blog Management System** (Version 1.0.0). It explains the system's features, behavior, and constraints for developers, testers, and other stakeholders.

The system allows users to register, log in, and manage blog content through a REST API. Registered users can create, update, and delete articles, and also add comments. Administrators have additional permissions to manage users and system content.

This document mainly covers the backend features of the system, including authentication, article management, comment management, and user management. It also describes important aspects such as security, performance, scalability, and deployment practices like caching, logging, testing, Docker, CI/CD, load balancing, and database replication.

1.2 Project Scope

The Blog Management System enables users to:

- Register and log in securely using authentication mechanisms (e.g., JWT-based auth with access and refresh tokens).
- Create, view, update and delete blog articles.
- View published articles and interact through comments.
- Allow administrators to manage users and control system-level resources with role-based access control (RBAC).
- Maintain secure sessions and support password change/reset and email verification workflows.

The system is built as a scalable RESTful API designed for production use, ensuring reliability, security, and performance. It includes features like caching for faster responses, structured logging for monitoring and debugging, and testing for quality assurance.

The architecture is enhanced with modern engineering practices such as Docker containerization, CI/CD pipelines, load balancing, and database replication to ensure scalability and high availability.

1.3 References

- Project guidelines provided by the client.
- OpenAPI Specification https://swagger.io/specification/
- Docker Documentation https://docs.docker.com
- Nodemailer Documentation https://nodemailer.com/about/
- JWT Authentication: https://jwt.io/

2. Overall Description

2.1 Product Perspective

The Blog Management System is a standalone backend application with several core components:

- **REST API:** Exposes endpoints for user authentication, article management, comment handling, and user administration, secured with JWT-based authentication and role-based access control (RBAC).
- **Authentication Module:** Handles user registration, login, email verification, and password reset workflows to ensure secure access to the system.
- **Content Management Module:** Manages blog articles and comments, allowing users to create, update, delete, and interact with posts.
- **User Management Module:** Provides administrative capabilities to manage user accounts, roles, and permissions within the system, including secure password change functionality for users.
- **Integration Layer:** Connects with external services such as MongoDB for data storage and Nodemailer for email delivery.

2.2 Product Features

The Blog Management System offers several main features that allow users to manage blog content through a RESTful API.

- **User Authentication and Authorization:** Users can register and log in using JWT-based authentication. The system supports email verification and provides an email-based password reset functionality for account recovery. It also implements role-

based access control to differentiate permissions between regular users and administrators.

- **Article Management:** Authenticated users can create, view, update and delete articles. Articles can also be retrieved with pagination, filtering, sorting, and searching.
- **Comment Management:** Users can add, view, update, and delete comments related to articles.
- **User Management:** Administrators can create users, view user details, update user information, delete users, and manage roles and account status. Users can change their own password also.
- **System Features:** The system also includes Redis caching, Winston logging, testing, Docker containerization, CI/CD pipelines, load balancing, and database replication to improve performance and scalability.

2.3 User Classes and Characteristics

The **Blog Management System** will be used by three main types of users:

- **End Users:** Guests and registered users who interact with blog content.
- **Administrators:** Manage users, content, roles, and system operations.
- **Developers:** Integrate and extend the system using APIs

2.4 Operating Environment

- **Backend:** Runs as a Node.js (Express.js) REST API with OpenAPI 3.0 documentation, deployed on development and production environments with load balancing support.
- **Database:** Uses MongoDB with optional replication for high availability.
- **Additional Components:** Includes Nodemailer (SMTP) for email verification, Redis for caching, Winston for logging, Docker for containerization, GitHub Actions for CI/CD, and Nginx (or similar) for load balancing and traffic distribution.

2.5 Design and Implementation Constraints

The Blog Management System has the following technical constraints:

- **Architecture:** System must follow a modular monolith architecture
- **API Design:** RESTful APIs must be used with separate public and admin routes.
- **Authentication:** JWT-based authentication with Role-Based Access Control (RBAC). Email verification and password reset must be handled using secure tokens.
- **Database:** MongoDB must be used as the primary database.
- **Security:** Passwords must be hashed (e.g., bcrypt).

- **Performance:** Pagination must be implemented for posts, comments, and other list-based data.

2.6 Assumptions and Dependencies

- **Assumptions:**
  - Users will have valid email addresses for registration and verification.
  - Client applications (web/mobile) will communicate via HTTP/JSON.
  - System users are assumed to follow standard authentication flow (login, token-based access).
- **Dependencies:**
  - Node.js (v18 or higher) as the runtime environment.
  - Express.js for building RESTful APIs.
  - MongoDB as the primary database.
  - Nodemailer or similar service for email verification and password recovery.
  - JWT for authentication and authorization.
  - Libraries such as bcrypt for password hashing and dotenv for environment configuration.

3. System Requirements

3.1 Functional Requirements

3.1.1 User Registration

- **Description:** Allow new users to create an account.
- **Input:** Name (string), email (string, valid format), password (string, minimum 8 characters).
- **Output:** Success message with user info or error.
- **Process:**
  - Validate input data.
  - Hash the password using bcrypt.
  - Store user data in the database.
  - Send verification email.
- **Constraints:** Email must be unique.

3.1.2 Email Verification

- **Description:** Verify user email to activate account.
- **Input:** Verification token (string).
- **Output:** Success message with JWT token or error message.

- **Process:**
  - Validate token.
  - Activate user account.
- **Constraints:** Token must be valid and not expired.

3.1.3 Resend Verification Email

- **Description:** Resend email verification link.
- **Input:** Email (string).
- **Output:** Success or error message.
- **Process:**
  - Validate email.
  - Generate new verification token.
  - Send email.
- **Constraints:** Only unverified users allowed.

3.1.4 User Login

- **Description:** Authenticate user using email and password.
- **Input:** Email (string), password (string).
- **Output:** Access token and refresh token or error message.
- **Process:**
  - Validate input data.
  - Verify email and password.
  - Generate JWT access token and refresh token.
- **Constraints:** Only verified and active users can log in.

3.1.5 Token Refresh

- **Description:** Generate a new access token using a valid refresh token.
- **Input:** Refresh token (HttpOnly cookie).
- **Output:** New access token or error message.
- **Process:**
  - Validate refresh token.
  - Generate new access token.
- **Constraints:** Refresh token must be valid and not expired.

3.1.6 Password Reset Request

- **Description:** Allow users to request a password reset link via email.
- **Input:** Registered email address (string).
- **Output:** Success message or error.
- **Process:**
  - Validate email.

    - Generate reset token.
    - Send password reset link via email.
- **Constraints:** Email must be registered in the system.

3.1.7 Password Reset Confirmation

- **Description:** Allow users to set a new password using a reset token.
- **Input:** Reset token (string), new password (string).
- **Output:** Success message or error.
- **Process:**
    - Validate reset token.
    - Hash new password.
    - Update password in database.
    - Invalidate reset token.
- **Constraints:** Token must be valid and not expired.

3.1.8 User Logout

- **Description:** Log out user and invalidate authentication session.
- **Input:** Refresh token (cookie).
- **Output:** Success message.
- **Process:**
    - Receive logout request.
    - Invalidate refresh token.
- **Constraints:** User must be authenticated.

3.1.9 Change Password

- **Description:** Allow users to update their account password.
- **Input:** User ID, old and new password (string).
- **Output:** Success message or error.
- **Process:**
    - Validate input data.
    - Verify old password.
    - Hash new password using bcrypt.
    - Update password in database.
- **Constraints:** User must be authenticated.

3.1.10 Initial Admin Setup

- **Description:** Create the first administrator account for the system.
- **Input:** Name (string), email (string), password (string).
- **Output:** Success message or error.
- **Process:**

    - Check if admin exists.
    - Create admin account.
    - Disable setup endpoint after first admin creation.
- **Constraints:** Can only be used once (first-time setup only).

3.1.11 Create Article

- **Description:** Allow authenticated users to create a new article.
- **Input:** Title (string), body (string), cover image (url).
- **Output:** Created article or error.
- **Process:**
    - Validate input data.
    - Verify user authentication.
    - Save article in database.
- **Constraints:** Only authenticated users can create articles.

3.1.12 View published Articles

- **Description:** Allow users to retrieve a list of articles.
- **Input:** Pagination parameters (page, limit), sorting, searching options.
- **Output:** List of articles or empty response.
- **Process:**
    - Fetch articles.
    - Apply filters, pagination.
- **Constraints:** Only published articles returned.

3.1.13 View Article Details

- **Description:** Retrieve single article.
- **Input:** Article ID, Expand options.
- **Output:** Article data or error.
- **Process:**
    - Retrieve articles from database.
    - Include related data if requested.
- **Constraints:** Article must exist. Public sees only published articles.

3.1.14 View Article Author

- **Description:** Retrieve author of article.
- **Input:** Article ID.
- **Output:** Author details or error.
- **Process:**
    - Fetch article.
    - Return author info.

- **Constraints:** Article must exist.

3.1.15 Update Article

- **Description:** Allow users to update their own articles.
- **Input:** Article ID, updated fields (title, body, cover image).
- **Output:** Success message or error.
- **Process:**
  - Validate input data.
  - Verify user authorization.
  - Update article in database.
- **Constraints:** Only the article owner can update.

3.1.16 Delete Article

- **Description:** Allow users to delete their own articles.
- **Input:** Article ID.
- **Output:** Success message or error.
- **Process:**
  - Verify user authorization.
  - Delete article from database.
- **Constraints:** Only the article owner or admin can delete.

3.1.17 Manage Article Status

- **Description:** Publish or hide article.
- **Input:** Article ID, Status.
- **Output:** Success message or error.
- **Process:**
  - Validate status.
  - Update article status.
- **Constraints:** Only admin can update the status of an article.

3.1.18 View All Articles (Admin)

- **Description:** Retrieve all articles.
- **Input:** Page, Limit, Filters.
- **Output:** List of articles.
- **Process:**
  - Authenticate admin.
  - Fetch all articles.
- **Constraints** Only admin can access this route.

3.1.19 Create Comment

- **Description:** Allow authenticated users to add comments to an article.
- **Input:** Article ID, comment text (string).
- **Output:** Created comment or error message.
- **Process:**
  - Validate input data.
  - Verify user authentication.
  - Store comment in database.
- **Constraints:** User must be authenticated. Article must exist.

3.1.20 View Article Comments

- **Description:** Allow users to view comments of a specific article.
- **Input:** Article ID, pagination.
- **Output:** List of comments or empty response.
- **Process:**
  - Fetch comments
  - Apply pagination
- **Constraints:** Only public comments visible. Article must exist.

3.1.21 Update Comment

- **Description:** Allow users to update their own comments.
- **Input:** Comment ID, updated text (string).
- **Output:** Success message or error.
- **Process:**
  - Validate input data.
  - Verify user authorization.
  - Update comment in database.
- **Constraints:** Only comment owner can update.

3.1.22 Delete Comment

- **Description:** Allow users to delete their own comments.
- **Input:** Comment ID.
- **Output:** Success message or error.
- **Process:**
  - Verify user authorization.
  - Delete comment from database.
- **Constraints:** Only comment owner or admin can delete.

3.1.23 Manage Comment Visibility

- **Description:** Hide or show comments.

- **Input:** Comment ID, Status.
- **Output:** Success message or error.
- **Process:**
  - Verify authorization.
  - Update comment status
- **Constraints:** Only admin can update status.

3.1.24 View All Comments (Admin)

- **Description:** Retrieve all comments.
- **Input:** Filters, Pagination.
- **Output:** List of comments or error.
- **Process:**
  - admin authorization.
  - Fetch comments.
- **Constraints:** Only admin can access this route.

3.1.25 Create User

- **Description:** Allow administrators to create new user accounts.
- **Input:** Name (string), email (string), password (string).
- **Output:** Created user or error message.
- **Process:**
  - Validate input data.
  - Verify admin authorization.
  - Create user account in database.
- **Constraints:** Only administrators can create users. Email must be unique.

3.1.26 View Users

- **Description:** Allow administrators to view all user accounts.
- **Input:** Filters, Pagination.
- **Output:** List of users or empty response.
- **Process:**
  - Verify admin authorization.
  - Retrieve users from database.
- **Constraints:** Only administrators can access user list.

3.1.27 View User Details

- **Description:** Retrieve single user.
- **Input:** User ID, Expand.
- **Output:** User data.
- **Process:**

    - Verify authorization.
    - Fetch user
- **Constraints:** User id must exist.

3.1.28 Update User Information

- **Description:** Allow administrators to update user details.
- **Input:** Name (string), role (string), status (string).
- **Output:** Success message or error.
- **Process:**
    - Validate input data.
    - Verify admin authorization.
    - Update user information in database.
- **Constraints:** Only administrators can update user data.

3.1.29 Delete User

- **Description:** Allow administrators to delete user accounts.
- **Input:** User ID.
- **Output:** Success message or error.
- **Process:**
    - Verify admin authorization.
    - Delete user from database.
- **Constraints:** Only administrators can delete.

3.2 Non-Functional Requirements

3.2.1 Performance

- **Response Time:** API responses should return within 1-2 seconds under normal load.
- **Scalability:** The system should handle up to 500-1,000 concurrent users.
- **Throughput:** The system should support at least 100–200 API requests per minute without degradation.

3.2.2 Security

- **Authentication:** JWT for authentication for APIs.
- **Data Protection:** Hash passwords with bcrypt. Encrypt sensitive data in transit (HTTPS).
- **Authorization:** Role-based access control to restrict access to resources based on user roles.

3.2.3 Reliability

- **Uptime:** 99.9% availability for APIs.
- **Data Integrity:** Ensure consistency of blog posts, comments, and user data during create/update/delete operations.
- **Error Handling:** Proper error handling and logging mechanisms should be implemented to prevent system crashes.

3.2.4 Usability

- **API Design:** APIs should follow RESTful principles with clear and consistent endpoint naming.
- **API Documentation:** Provide API documentation using Swagger/OpenAPI.
- **Developer Experience:** Include clear request/response examples and error messages for easier integration.

3.2.5 Maintainability

- **Modularity:** Follow a modular monolith architecture.
- **Code Quality:** Enforce ESLint and Prettier for consistent code formatting and quality.
- **Testing:** Ensure at least 70–80% unit test coverage for core business logic.
- **Version Control:** Maintain clean Git workflow with proper commit structure.

3.2.6 Portability

- **Deployment:** The system should be deployable on any cloud platform supporting Node.js (e.g., Docker-based environments).
- **Database:** Compatible with MongoDB local and cloud instances.

3.3 Interface Requirements

3.3.1 REST API

- **Endpoints**
  - POST /auth/sign-up: Register a new user
  - POST /auth/setup-admin: Create initial admin account
  - POST /auth/sign-in: Log in and receive access token
  - POST /auth/refresh: Refresh access token
  - POST /auth/logout: Log out user
  - GET /auth/verify-email/{token}: Verify email address
  - POST /auth/resend-verification: Resend verification email
  - POST /auth/forgot-password: Request password reset
  - PATCH /auth/reset-password/{token}: Reset password

- POST /articles: Create a new article
- GET /articles: List published articles
- GET /articles/admin: List all articles (admin)
- GET /articles/{id}: Get article details
- PUT /articles/{id}: Create or fully update article
- PATCH /articles/{id}: Partially update article
- DELETE /articles/{id}: Delete article
- GET /articles/{id}/author: Get article author
- GET /articles/{id}/comments: List article comments
- POST /articles/{id}/comments: Create comment for article
- GET /comments: List all comments (admin)
- POST /comments: Create a comment
- PATCH /comments/{id}: Update comment
- DELETE /comments/{id}: Delete comment
- POST /users: Create a user (admin)
- GET /users: List all users
- GET /users/{id}: Get user details
- PATCH /users/{id}: Update user
- DELETE /users/{id}: Delete user
- PATCH /users/{id}/change-password: Change user password

4. System Architecture

4.1 Project Structure

The system follows a modular monolithic architecture where all components are organized within a single backend application.

- **server/**: Root backend application directory
  - **src/**: Contains the core source code
    - **api/**: Handles request processing and controllers
    - **routes/**: Defines API routes
    - **model/**: Contains database models (Mongoose schemas)
    - **db/**: Database connection configuration
    - **middleware/**: Custom middleware (auth, error handling)
    - **config/**: Application configuration
    - **lib/**: Shared services (e.g., token, email)
    - **utils/**: Helper functions
    - **app.js**: Express app configuration
    - **index.js**: Application entry point

- .env: Environment variables
- swagger.yaml: API documentation
- package.json → Dependencies and scripts
- .eslintrc.json → ESLint configuration

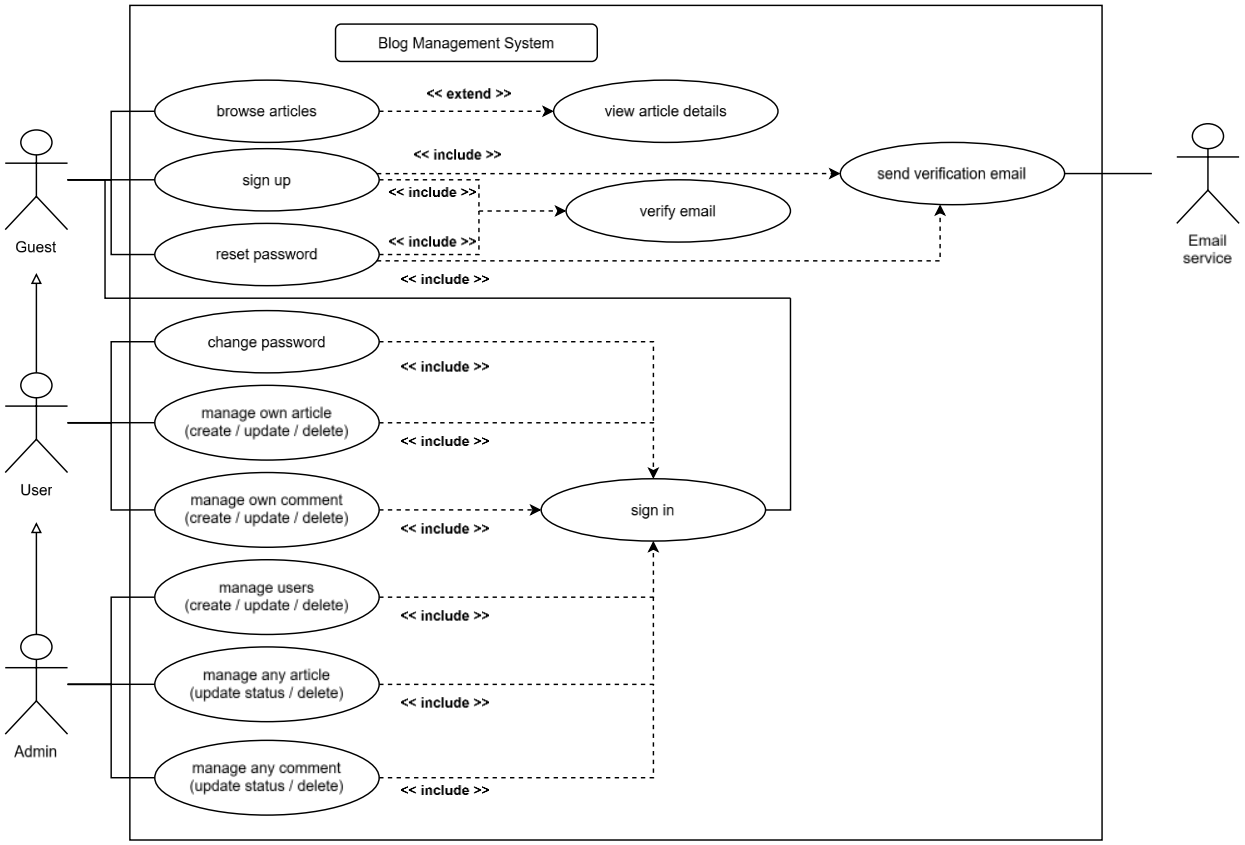
4.2 Technology Stack

- **Backend:** Node.js, Express.js.
- **Database:** MongoDB, Mongoose (ODM).
- **Authentication & Security:** JWT, bcrypt, HttpOnly Cookies.
- **Email Service:** Nodemailer with SMTP.
- **API Documentation:** OpenAPI (Swagger YAML).
- **Development Tools:** ESLint, Yarn, Prettier
- **Deployment Tools:** Docker and CI/CD tools for automated build and deployment.
- **Caching:** Redis.
- **Logging:** Winston for recording system events and errors.

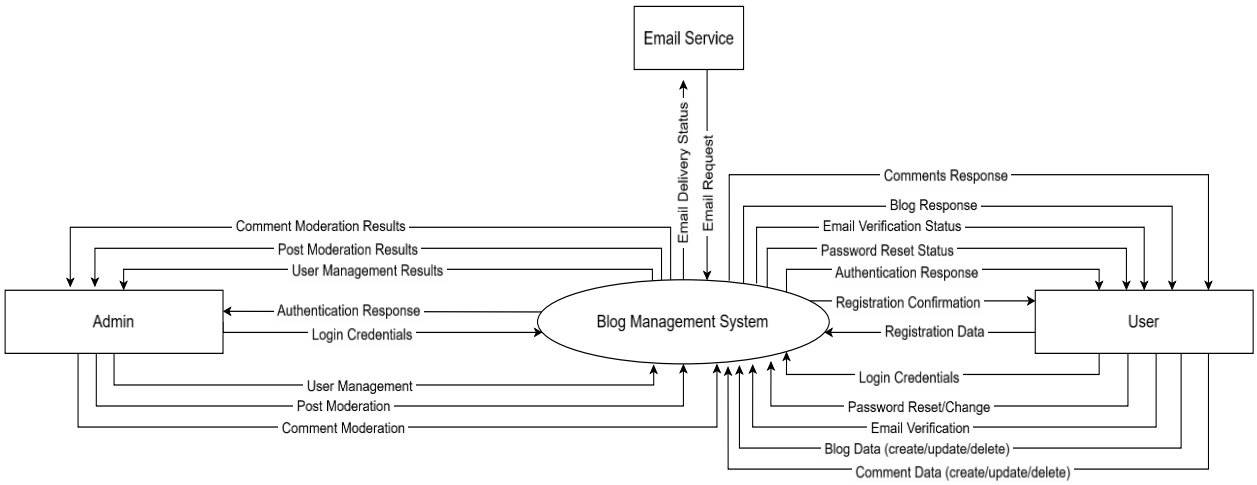
Appendix A: Glossary

- **API:** Application Programming Interface.
- **JWT:** JSON Web Token used for authentication.
- **CI/CD:** Continuous Integration / Continuous Deployment.
- **SMTP:** Simple Mail Transfer Protocol used for sending emails.
- **REST:** Representational State Transfer architecture for web services.

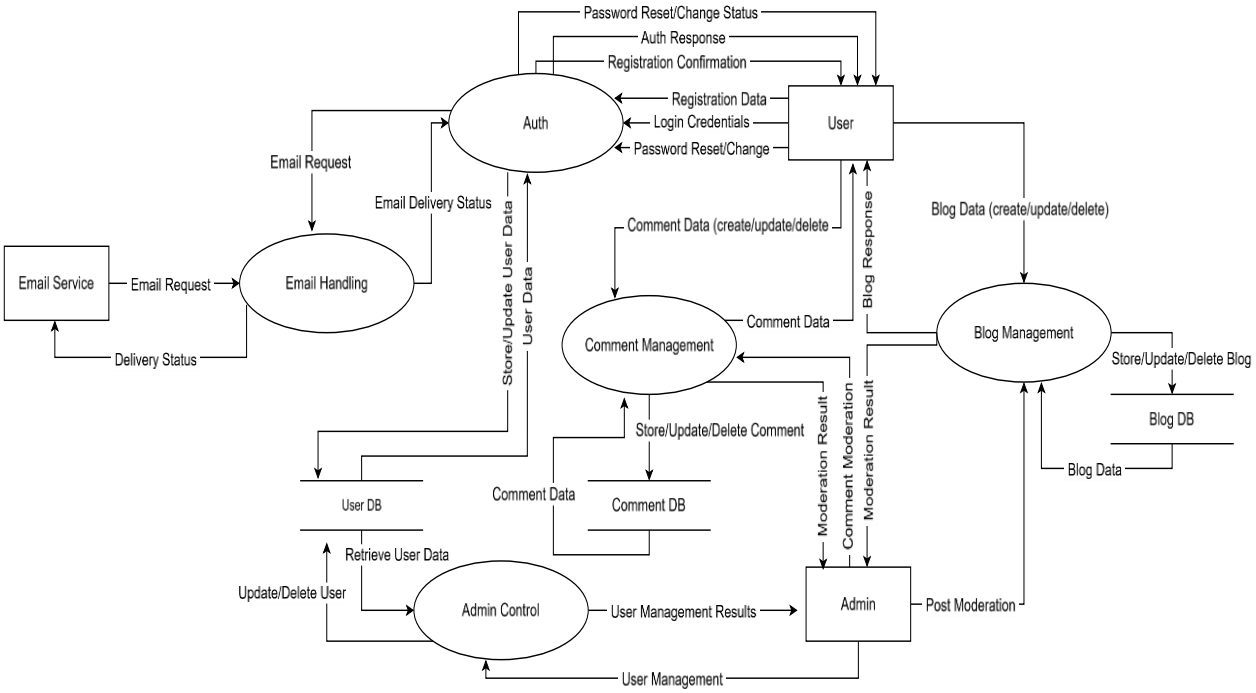
Appendix B: Analysis Models



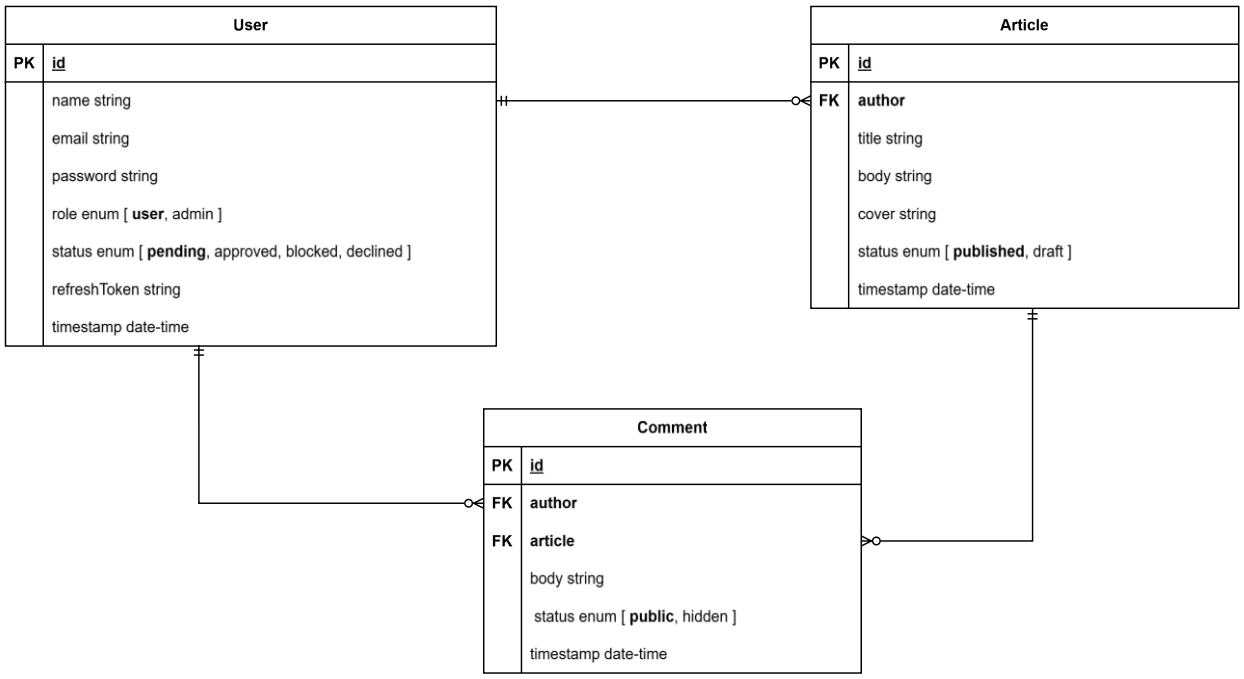
Use Case Diagram



Level 0 Data Flow Diagram



Level 1 Data Flow Diagram



Entity Relationship Diagram